Guião · Matilde

Notícias, criador de mercado, broker, ecrã inicial, dashboard e gráficos finais

SAD · Grupo 2 · Volatility Interruptions and Crash Contagion in Euronext Lisbon · entrega a 2 de dezembro. Os nomes vêm de `docs/CONTRATO.md`.

Resumo das tuas semanas

Semana	Entrega concreta
1 (17–24 out)	Tutorial de gráficos e controlos; esboço do ecrã inicial e do dashboard; statecharts em papel. Quando o modelo base sair: começar o NewsAgent.
2 (24–31 out)	**NewsAgent e MarketMaker a funcionar.** Commit e avisar.
3 (31 out–7 nov)	Ecrã inicial (sliders) e cores dos agentes por estado.
4 (7–14 nov)	**Dashboard final.**
5 (14–21 nov)	Fase 2: **Broker completo** (fila + dois statecharts).
6 (21–28 nov)	**Gráficos finais** (a partir do kpis.xlsx do João), apresentação e a tua parte do relatório (interface, criador de mercado, notícias, broker).

0. Como trabalhar (lê isto primeiro)

Uma vez só

1. Aceita o convite do GitHub que chega ao teu email (repositório **sad-euronext-g2**).
2. Instala o **GitHub Desktop** (desktop.github.com) e entra com a tua conta.
3. No GitHub Desktop: **File → Clone repository → sad-euronext-g2 → Clone**. Fica uma pasta no teu PC (por defeito em Documentos\GitHub\sad-euronext-g2).
4. Confirma com o grupo que **todos usam a mesma versão do AnyLogic** (Help → About). Um modelo gravado numa versão mais recente pode não abrir numa mais antiga.

Todos os dias

1. GitHub Desktop: **Fetch origin → Pull** (trazes o que os outros fizeram).
2. AnyLogic: **File → Open** e abres o teu `.alp` dentro da pasta do repositório. Não reinstalas nada: o GitHub só mantém a pasta sincronizada.
3. Trabalhas e gravas (Ctrl+S).
4. GitHub Desktop: escreves uma mensagem curta (ex.: `João: fundamentalista com statechart`) → **Commit to main → Push origin**.

Onde trabalhas

- `model/Euronext_Main.alp` é o modelo principal e **só o Miguel o edita**.
- Tu trabalhas em `sandbox/matilde/`. Depois do Miguel publicar o modelo base, copia `model/Euronext_Main.alp` para a tua pasta com outro nome (ex.: `sandbox/matilde/matilde.alp`) e abre esse. Na tua cópia podes mexer em tudo, incluindo no Main.
- Quando a tua peça funcionar: commit + push e avisa no grupo. O Miguel abre os dois modelos no AnyLogic e, no painel **Projects**, copia o teu agente (Ctrl+C) e cola-o no modelo principal (Ctrl+V). Isto só funciona sem erros se usares os nomes do `docs/CONTRATO.md`.
- Se estragares a tua cópia: GitHub Desktop → separador **History** → botão direito num commit antigo → **Revert**. Ou voltas a copiar o modelo principal.

AnyLogic em 1 minuto

Coisa	Para que serve	Onde está
Agent type	um tipo de participante (ex.: `Fundamentalist`)	Palette → Agent → arrastar **Agent**
Population	muitos agentes do mesmo tipo dentro do Main	assistente do Agent → *Population of agents*
Parameter	valor fixo que muda de cenário para cenário (ex.: `nChartists`)	Palette → Agent → Parameter
Variable	valor que muda durante a simulação (ex.: `cash`)	Palette → Agent → Variable
Function	bloco de código que se chama pelo nome	Palette → Agent → Function
Event	código que corre num instante ou de X em X minutos	Palette → Agent → Event
Statechart	os estados de um agente e o que o faz mudar de estado	Palette → Statechart
Process Modeling Library	blocos de filas (Enter, Queue, Hold, Delay, Sink…)	Palette → Process Modeling Library
`main`	dentro de qualquer agente, `main` dá acesso ao Main (ex.: `main.stock.mid()`)	—

Como desenhar um statechart

1. Abre o agente: duplo clique no tipo (ex.: `Fundamentalist`) no painel **Projects**.
2. Palette → **Statechart** → arrasta **Statechart Entry Point**. O nome fica `statechart`; não o mudes (o Miguel chama `statechart` em todos os agentes).
3. Arrasta um **State**, encosta-lhe a ponta do Entry Point e dá-lhe o nome em **Name**.
4. Transição: na Palette faz **duplo clique em Transition** (fica em modo de desenho), clica na borda do estado de origem e depois na borda do estado de destino.
5. Clica na transição e escolhe **Triggered by**: **Rate** (acontece em média X vezes por minuto, ao acaso: "de vez em quando decide"); **Timeout** (X minutos depois de entrar no estado; com Timeout 0 o agente faz a ação e volta logo); **Message** (quando chega uma mensagem: Message type `String`, Fire transition *On particular message*, Message `"HALT"`); **Condition** (usada à saída de um Branch).
6. **Branch** (o losango na Palette Statechart) serve para decidir. Cada seta que sai dele tem uma Condition; uma tem de ser **Default** (a que se segue se nenhuma outra for verdadeira).
7. Código: cada estado tem **Entry action** (corre ao entrar) e cada transição tem **Action** (corre ao passar) e **Guard** (condição extra para a transição poder acontecer).
8. **Estado composto**: desenha um State grande e arrasta outros para dentro; dentro dele põe um **Initial State Pointer** apontado para o primeiro estado. Uma seta que sai do estado grande vale para todos os de dentro: é assim que se faz o `HALT` (de qualquer estado, o agente vai para `Halted`).
9. Testar: carrega em Run; na barra de cima da janela da simulação escolhe o agente (ex.: `fundamentalists[0]`) e vês o estado ativo a verde. Para escrever na consola: `traceln("texto " + x);`.

Este guião tem o código para colar. Onde aparece **Entry action**, **Action** ou o nome de uma função, copia o bloco cinzento para esse campo. Se o AnyLogic não aceitar `statechart.receiveMessage(msg)`, usa `statechart.fireEvent(msg)`, que faz o mesmo.

1. Semana 1

1. Tutorial (~2 h): no AnyLogic, Help → **AnyLogic Help** → procura “Time Plot”, “Bar Chart”, “Slider” e “Radio buttons”. Em cada página, faz o exemplo num modelo de teste qualquer.
2. Em papel: ecrã inicial e dashboard (esboço abaixo; podes melhorar).
3. Lê o capítulo 0 e desenha em papel os statecharts do NewsAgent e do MarketMaker (secções 2 e 3).
4. Quando o Miguel publicar o modelo base: Pull → copia `model/Euronext_Main.alp` para `sandbox/matilde/matilde.alp` → abre esse.

Plano B: se o modelo base ainda não estiver pronto na quinta

Não esperes. Cria um modelo de teste teu (`sandbox/matilde/teste.alp`) com um **Stock falso**, que tem os mesmos nomes do verdadeiro. Os teus agentes funcionam com ele, e quando o modelo base sair só copias os teus agentes para lá (Ctrl+C / Ctrl+V no Projects).

1. Novo modelo (time units: minutes). No Main: parâmetro `initialPrice` 0.5, variáveis `fundamentalValue` (Initial value `initialPrice`) e `nextOrderId` long 0, e uma função `broadcast(String msg)` vazia.
2. **Single agent** `stock`, do tipo `Stock`. Variáveis `lastPrice` e `ma` (Initial value `main.initialPrice`) e `halted` false.
3. Funções do Stock falso:

```
// mid (Returns double)
return lastPrice;

// order (Just action; trader Agent, side int, price double, qty int): o preço mexe um pouco no sentido da ordem
lastPrice *= 1 + side * 0.0002 * qty / 1000.0;
traceln(time() + " ordem " + (side == 1 ? "COMPRA " : "VENDA ") + qty + " @ " + price);

// cancelAll (Just action; a Agent)
// vazio
```

Event `walk` no Stock falso (Rate 1): `lastPrice *= 1 + normal(0.002, 0); ma = 0.98 * ma + 0.02 * lastPrice;`

```
ECRÃ INICIAL (experiência Simulation)          DASHBOARD (Main)
+--------------------------------------+       +---------------------------+------------------+
| Fundamentalistas [=====|----]   40   |       | Preço vs valor fundamental| Estado: CONTÍNUO |
| Tendência        [===|------]   30   |       | (time plot)               | Interrupções: 3  |
| Ruído            [===|------]   30   |       |                           | Queda máx: 12.4% |
| Colares ( ) nenhum ( ) 2/5 (x) 3/8   |       +---------------------------+ Volume: 1.2M     |
|         ( ) 5/10                     |       | Spread (time plot)        +------------------+
| Pausa [ 3] min   Pausa longa [10] min|       |                           | Livro: compras / |
| Choque [x]   tamanho [==|-----] -10% |       +---------------------------+ vendas (barras)  |
|               [ RUN ]                |       | Histograma de retornos    | Agentes (cor =   |
+--------------------------------------+       |                           |  estado)         |
                                               +---------------------------+------------------+
```

2. NewsAgent: notícias e choques (semana 2)

Gera a informação externa. Em geral está calmo e de vez em quando há pequenas notícias. No cenário de choque, no dia 60, o valor fundamental cai (ex.: −10%, como o BES) e todos são avisados com `SHOCK`; segue-se uma fase de dissipação com pequenas réplicas.

1. Parâmetros no Main: `shockOn` boolean false, `shockTime` 30600, `shockSize` -0.10, `newsRate` 0.0004.
2. Palette → Agent → **Agent** → **Single agent** → type `NewsAgent`, name `news`.
3. Event `shockEvent`: Trigger *Timeout*, Mode **Occurs once**, Occurrence time `main.shockTime`. Action: `if (main.shockOn) statechart.receiveMessage("BIG_SHOCK");`

```
Calm --Rate newsRate--> SmallNews --T0--> Calm
Calm --Message "BIG_SHOCK"--> BigShock --Timeout 30--> Fading --Timeout 120--> Calm
                                                        Fading --Rate 0.05--> Fading  (réplicas)
```

```
// SmallNews -> Entry action (notícia pequena, boa ou má)
main.fundamentalValue *= 1 + normal(0.01, 0);

// BigShock -> Entry action
main.fundamentalValue *= 1 + main.shockSize;
main.broadcast("SHOCK");
traceln("CHOQUE em t=" + time());

// Fading -> Fading (transição para o próprio estado, Rate 0.05) -> Action
main.fundamentalValue *= 1 + normal(0.005, -0.003);     // pequenas réplicas, em média negativas
```

Teste: `shockOn = true` e `shockTime = 600`. Ao minuto 600 deve aparecer "CHOQUE" na consola e o `fundamentalValue` deve cair no gráfico (os fundamentalistas do João arrastam depois o preço).

3. MarketMaker: criador de mercado (semana 2)

Dá liquidez: põe sempre uma compra um pouco abaixo e uma venda um pouco acima do preço médio e atualiza-as a cada minuto. Durante a reserva retira as ordens.

1. Parâmetros no Main: `mmSpread` 0.004, `mmQty` int 2000.
2. **Single agent** → type `MarketMaker`, name `marketMaker`. Variáveis: `cash` double 0, `position` int 0.
3. Função **quote** (Just action):

```
main.stock.cancelAll(this);                    // tira as cotações antigas
double m = main.stock.mid();
main.stock.order(this, 1, m * (1 - main.mmSpread / 2), main.mmQty);
main.stock.order(this, -1, m * (1 + main.mmSpread / 2), main.mmQty);
```

```
Quoting (entry: quote();) --Timeout 1--> Quoting      (seta do estado para ele próprio)
Quoting --Message "HALT"--> Paused (entry: main.stock.cancelAll(this);) --Message "REOPEN"--> Quoting
```

A seta de Quoting para Quoting: com a Transition em modo de desenho, clica na borda de Quoting e outra vez noutro ponto da borda de Quoting. Ao re-entrar, o Entry action corre de novo, por isso volta a cotar a cada minuto.

4. Ecrã inicial (semana 3)

1. Projects → `Simulation` → Properties → botão **Create default UI**. O AnyLogic cria um controlo para cada parâmetro, já ligado.
2. Apaga os que não interessam ao utilizador. Ficam: `nFundamentalists`, `nChartists`, `nNoise` (Slider 0–100), `collarSet`, `haltShort`, `haltLong` (Edit box), `shockOn` (Check box), `shockSize` (Slider −0.30 a 0).
3. Para `collarSet` apaga o controlo criado e põe um **Radio Buttons** (Palette Controls) com 4 opções: "Sem colares", "2% / 5%", "3% / 8% (atual)", "5% / 10%". Em **Link to** escolhe o parâmetro `collarSet`: a 1.ª opção vale 0, a 2.ª vale 1, etc.
4. Título, logótipo da FEUP (Palette Presentation → **Image**) e uma frase a explicar o modelo.

5. Dashboard (semanas 3–4)

Faz no Main da tua cópia; depois o Miguel copia os elementos para o modelo principal (copiar e colar formas entre modelos funciona).

Elemento	Palette	O que pôr
Preço	Analysis → Time Plot	Values `stock.lastPrice` ("Preço") e `fundamentalValue` ("Valor fundamental"); Time window 510

Elemento	Palette	O que pôr
Spread	Analysis → Time Plot	Value `stock.spread()`
Livro de ordens	Analysis → Bar Chart	Values `stock.bidBook.size()` (“Compras”) e `stock.askBook.size()` (“Vendas”)
KPIs	Presentation → Text	Text dinâmico (clica no ícone = ao lado do campo): `"Estado: " + (stock.halted ? "RESERVA" : "CONTÍNUO")`, `"Interrupções: " + nHalts`, `String.format("Queda máxima: %.1f%%", 100 * maxDrawdown)`, `"Volume: " + stock.volume`
Retornos	Analysis → Histogram Data + Histogram	Histogram Data `retData` (40 intervalos); Histogram com Data `retData`

Para o histograma: variável `prevPrice` no Main (Initial value `initialPrice`) e um Event `retEvent` (Cyclic, 1 min):

```
retData.add(Math.log(stock.lastPrice / prevPrice));
prevPrice = stock.lastPrice;
```

Cores dos agentes por estado

1. Main → Properties → **Space and network** → Layout type *Random*.
2. Abre o tipo `Fundamentalist` (do João) → Palette Presentation → **Oval** pequeno no centro → Fill color → dinâmico: `inState(Buying) ? green : inState(Selling) ? red : inState(Halted) ? gray : silver`
3. Igual no `Chartist`: `inState(Panic) ? red : inState(TrendSell) ? orange : inState(TrendBuy) ? green : silver`
4. Durante a simulação, o livro de ordens anima-se sozinho: escolhe `stock` na barra de cima e vês as filas a encher e a esvaziar.

6. Fase 2: Broker (semana 5)

O MarketMaker em stress (Fase 2) fica com o João: ele acrescenta estados ao teu agente.

Broker, parte 1: a fila (Process Modeling Library)

O broker recebe as ordens dos investidores alavancados, põe-nas numa fila com capacidade limitada e envia-as para o mercado. Se a plataforma estiver sobrecarregada, rejeita ordens. É a “fila do broker” da proposta.

1. Parâmetros no Main: `brokerRate` 2 (ordens processadas por minuto, por servidor), `brokerServers` int 1, `graceMinutes` 15.
2. Palette → Agent → **Agent** → **Population of agents** → type `Broker`, population `brokers`, Initial number 1. Parâmetro do Broker: `capacity` int 50. Variáveis: `accepting` boolean true, `rejectedCount` int 0.
3. Dentro do Broker, desenha com a Process Modeling Library:

```
brokerIn --> tmStart --> accept --(true)--> brokerQueue --> processing --> tmEnd --> toMarket
                         '--(false)--> rejected (Sink)
```

Bloco	Tipo	Propriedades (em todos: Agent type `Order`)
brokerIn	Enter	—
tmStart / tmEnd	TimeMeasureStart / TimeMeasureEnd	no tmEnd, TimeMeasureStart: `tmStart`. O KPI do tempo de espera é `tmEnd.distribution.mean()`
accept	SelectOutput	Condition `accepting && brokerQueue.size() < capacity`

Bloco	Tipo	Propriedades (em todos: Agent type `Order`)
brokerQueue	Queue	Maximum capacity
processing	Delay	Delay time `exponential(main.brokerRate)`; Capacity `main.brokerServers`
toMarket	Exit	On exit: `main.stock.enter.take(agent);`
rejected	Sink	On enter: `rejectedCount++;`

Função **order** do Broker (Just action; argumentos `trader` Agent, `side` int, `price` double, `qty` int). É igual à do Stock, mas a ordem entra no broker:

```
Order o = new Order();
o.id = main.nextOrderId++;
o.side = side;
o.price = main.roundTick(price);
o.qty = qty;
o.tSubmit = time();
o.trader = trader;
brokerIn.take(o);
```

Teste: põe `capacity = 5` e `brokerRate = 0.2`: o `rejectedCount` deve subir e a fila deve encher na animação.

Broker, parte 2: dois statecharts em paralelo

No mesmo agente `Broker` põe **dois** Statechart Entry Points: `riskChart` e `platformChart`. O `platformChart` liga e desliga a variável `accepting` que a fila usa.

```
platformChart:  Operational --Timeout 1--> <load>
                   brokerQueue.size() > 0.95 * capacity --> Rejecting   (entry: accepting = false;)
                   brokerQueue.size() > 0.70 * capacity --> Overloaded  (entry: accepting = true;)
                   default                              --> Operational (entry: accepting = true;)
                Overloaded / Rejecting --Timeout 1--> <load>

riskChart:      Normal --Message "MARGIN_CALL"--> MarginAlert --Timeout main.graceMinutes--> ForcedLiquidation
                ForcedLiquidation (entry: manda "LIQUIDATE" aos clientes) --Timeout 30--> Normal
```

`graceMinutes` é o "regras de margem apertadas vs. folgadas" da proposta (ex.: 5 vs. 60 min). O LeveragedTrader da Margarida manda `MARGIN_CALL` ao broker quando entra em MarginAlert. Para a mensagem chegar ao statechart certo: Broker → Properties → **On message received**: `riskChart.receiveMessage(msg);`

7. Semana 6: gráficos finais

O João dá-te o `data/processed/kpis.xlsx` final. Fazes estes gráficos, que vão para o relatório e para a apresentação:

- Preço com e sem colares no mesmo choque (mesma semente): mostra o efeito da interrupção. Corre a Simulation duas vezes com `collarSet` 2 e 0 e exporta o Time Plot (botão direito → Copy all → Excel).
- Boxplot da queda máxima por `collarSet` (folha **runs**, 30 replicações cada).
- **Trade-off estabilidade vs. liquidez**: queda máxima (eixo x) contra volume ou tempo em interrupção (eixo y), um ponto por combinação da grelha (folha **resumo**). É a figura central da proposta.
- Volatilidade e spread antes e depois, a 2, 5 e 10 min (folha **halts**): gráfico de barras.
- Podes fazê-los em Excel, ou pedir ao Claude Code um script Python que os faça todos a partir do kpis.xlsx.

8. Semana 6: apresentação

- Estrutura sugerida (10–12 slides): problema (PSI, BES 2014, março 2020) → colares da Euronext → livro de ordens como fila → agentes (um slide com os statecharts) → dados e calibração (tabela BCP real vs. modelo) → cenários → resultados (o trade-off) → conclusões e limitações.
- Uma demonstração ao vivo de 1 minuto do dashboard com o choque ligado costuma resultar muito bem.